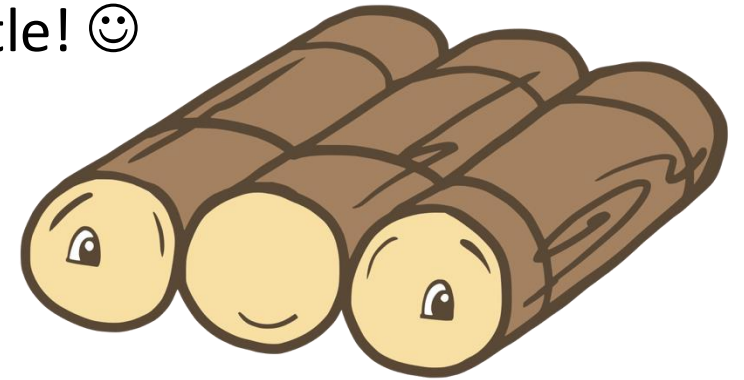


2PC and 3PC (consensus!)

BIG ideas! Will take a while to settle! 😊



Reading!

This [CLASSIC Tutorial from GoogleLinks to an external site.](#) (an oldie but a goodie?!)...

Load testing? VERY *applicable*, LITTLE'S LAW!



$$L = \lambda W$$

L – Average number of items within the system

λ – Average arrival rate of items into and out of the system

W – Average time an item spends in the system

Kanban and Little's Law. <https://kanbanzone.com/resources/lean/littles-law/>

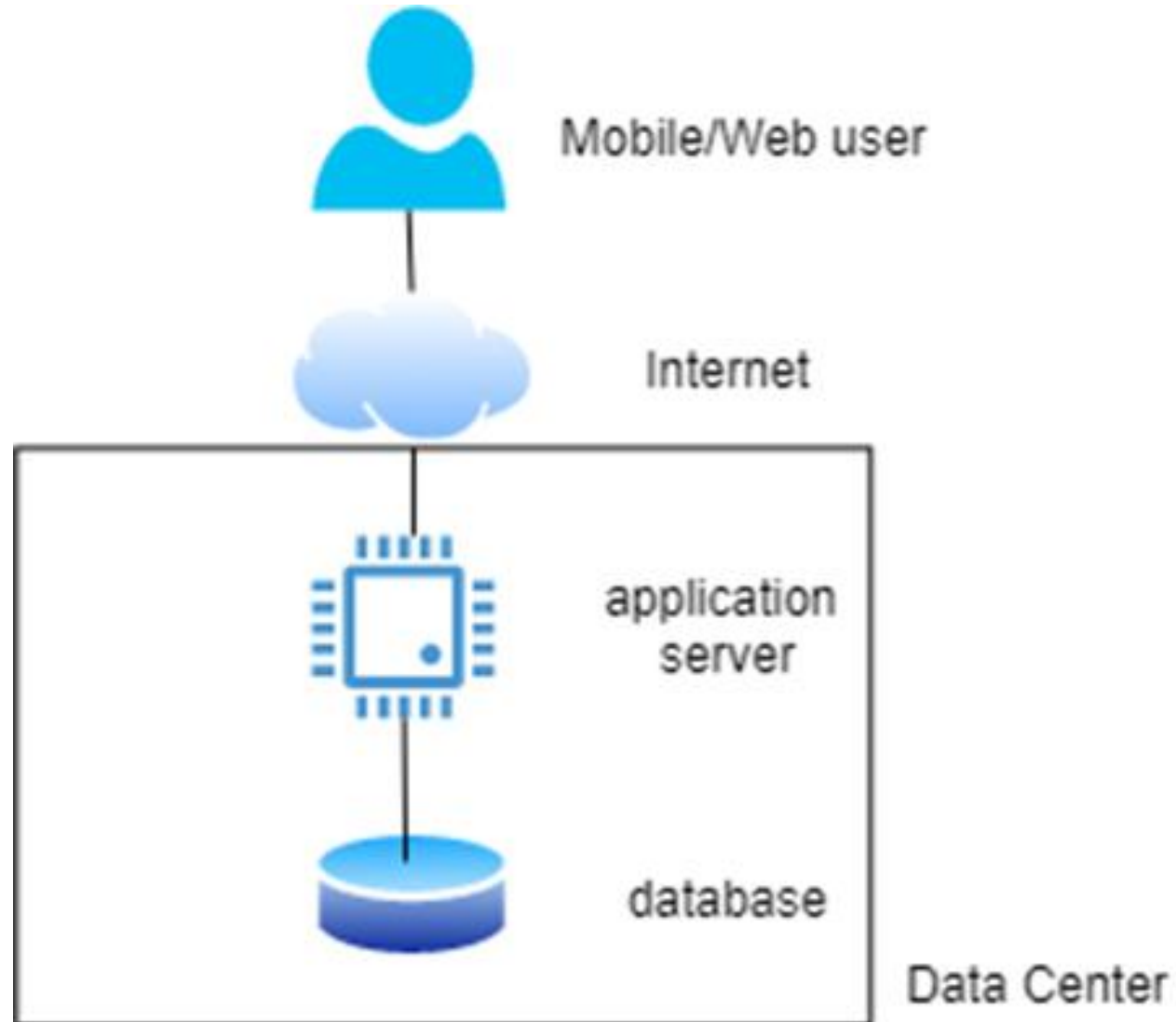
High Level Architecture Considerations

- User Functionality before Scalability
- Multi-tier Architecture
- Scale-up and Scale-out
- Load Balancing
- Stateless Services
- Distributed Databases
- Queuing

Software Systems Origins

- Very few systems start with scalability as the primary driver
- Agile methods prioritize delivering user functionality
 - **YAGNI** (“You Aren’t Gonna Need It”) principle
- Rapid application delivery frameworks are common
 - Ruby on Rails, Django, Apache/PHP, Two-tier architectures etc
- Trade off ease of delivery vs performance, scalability and availability

Basic Three-tier Distributed Architecture



Application Server

- Execution platform providing environment for application services (APIs)
 - Kubernetes
 - Cloudscape
- Accepts client requests and invokes appropriate service functionality
 - Based on API request endpoint/parameters
 - Endpoint is a URL with parameters
- **Multithreaded execution environment** using a thread pool
- Connections to database managed through a connection pool
 - Thread pool size > connection pool size
- Logging
- Monitoring

Scale Up

- As request loads grow, scale up the system to support high load
- Get a bigger application server and database server
 - More CPUs
 - More memory
 - More disks
 - More rack space
 - Additional costs

Scale Up – Pros and Cons

- Advantages

- Simple to replace a low-end server with high-end server

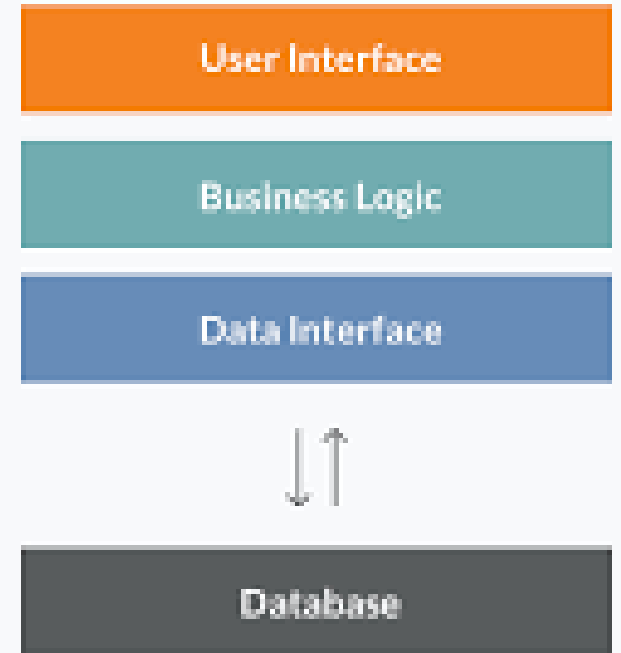
- Disadvantages

- Larger machines are more expensive per compute capacity
 - Single Point of Failure (SPoF)
 - Highly reliable hardware is available but expensive.
 - There is still a limited capacity
 - No matter how big the server is the capacity is still limited
 - Eventually you can't buy a bigger machine – you have the biggest available

Monoliths are Ponderous

- Implementations for 10s-100s of APIs all bundled in same code
- Hard to make changes and evolve
 - Big companies don't like scheduled downtime
- One alternative is **microservices** – lightweight and popular

Monolithic Architecture



What is a Microservice?

A microservice is a small, independent part of an application that communicates with other services through APIs. Microservices are used to build applications that are more scalable, resilient, and faster to develop.

- Feels like a Remote Procedure Call
- Loosely Coupled
 - Allows parallel development
- Not necessarily RESTful

Scale Out

- Replicate a service and run multiple copies on multiple server nodes
 - Use “cheap” commodity hardware
 - Cloud companies have custom servers, but still cheaper than a giant supercomputer
 - Buy a whole rack at-a-time, so “the rack is the computer”
- Requests from clients are distributed across the replicas
- If we have N replicas, each server node processes $\{\text{\#requests}/N\}$
 - Adding processing capacity as simple as adding a new server node

Load Balancers

- Load Balancer
 - Distributes requests across available services
 - Load balancing policy (e.g. round robin)
- Stateless services
 - Each request must be self-contained so load balancer can choose any service
 - Any session state must be stored **external to a service**
 - Example :- last page visited, shopping cart, session state



Scale-Out Advantages

- Add new (virtual) services to handle increased request loads
 - As latencies rise, simply deploy another server instance
 - Requires no code changes
 - Just pay for the hardware actually used
- Availability: If a service fails, the requests it is processing will be lost
- Failed service manages no session state
 - requests can be simply reissued by the client and sent to another service instance for processing

Database Bottleneck

- Adding new service instances gives increased capacity
- Eventually increased capacity will overload the database
- Slow queries will mean longer response times
- If overload continues, some system component will fail due to resource exhaustion
 - clients will see exceptions and request timeouts
- Need to remove **database bottleneck**

Options to Scale The Database

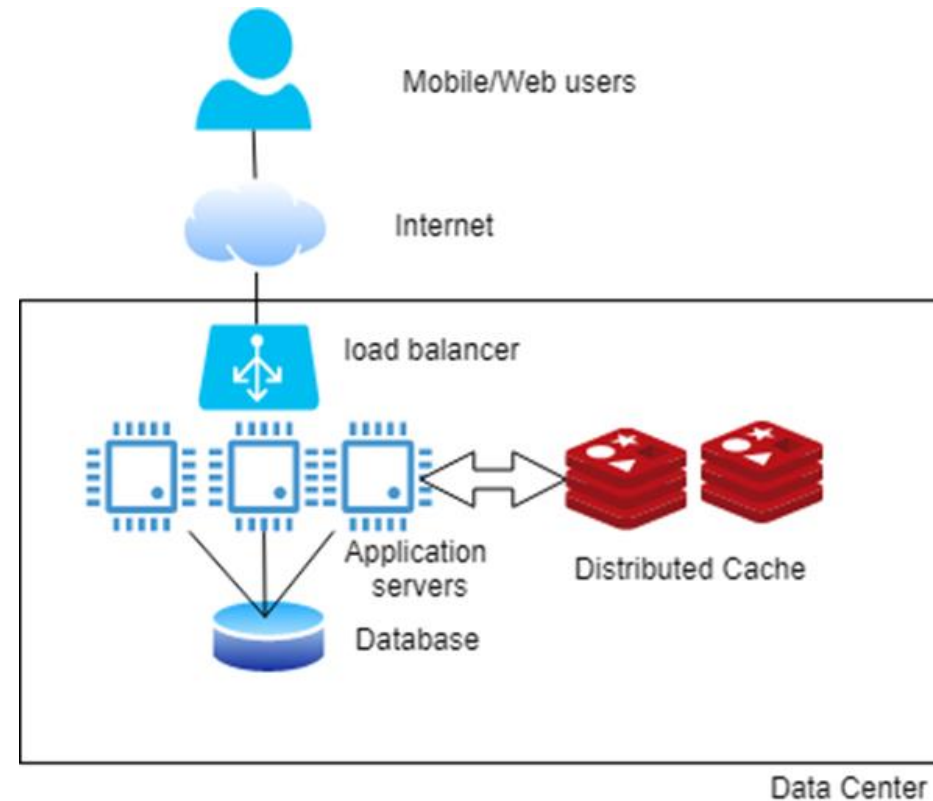
- Simple to scale up but the problem of SPoF still persists
- GCP can provision a SQL database on 96 vCPUs, 624GB of memory, 30TBs of disk and can support 4000 connections
- Cost between \$6K and \$16K per year
- How can we improve this?

Caching of Database Results

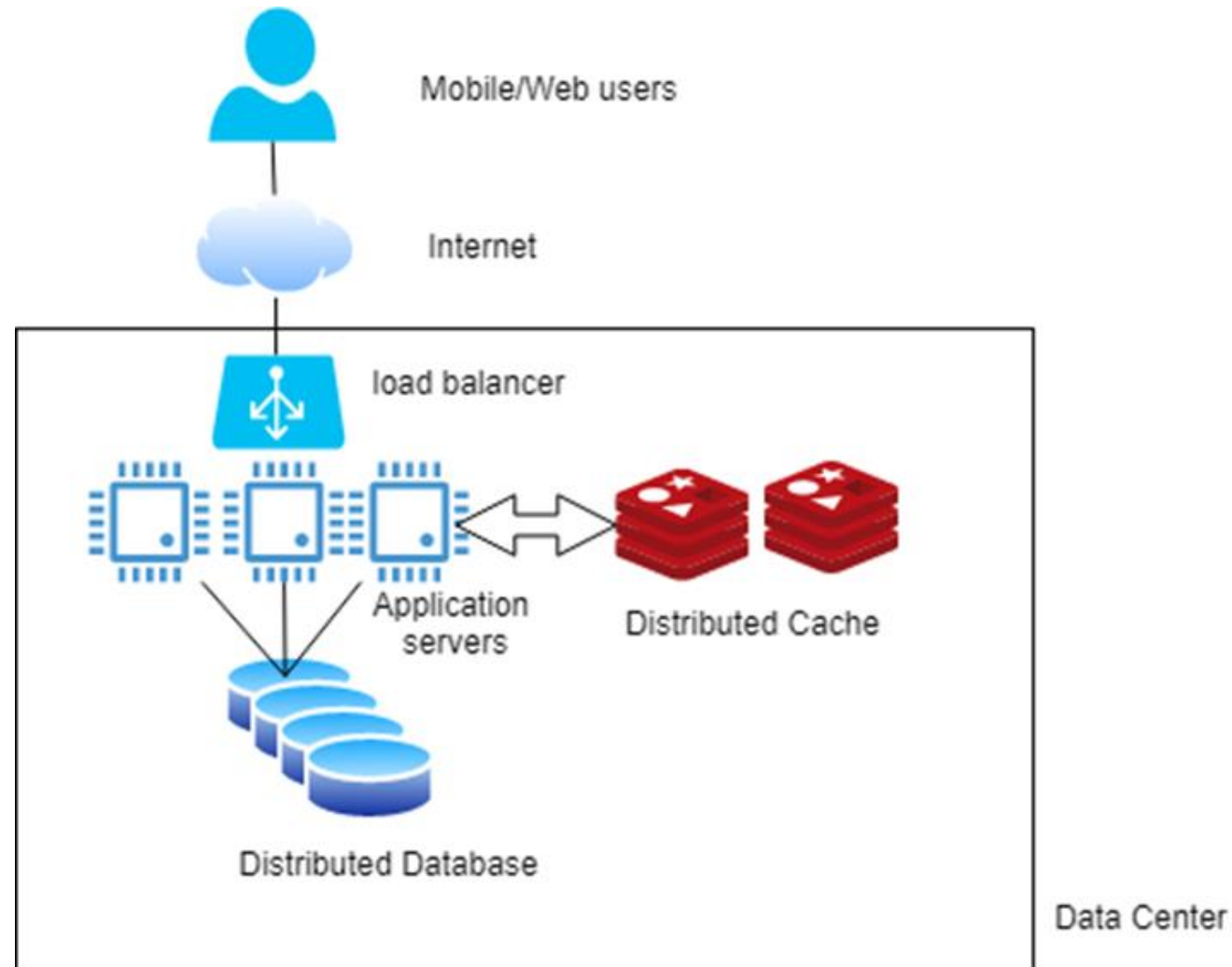
- Avoid using the database!!
- Employ distributed caching in the service tier
- Caches store commonly accessed database results in memory
- Can be **quickly** retrieved without database access
- Distributed caches such as a **Redis** or **memcached** widely used
 - Distributed Key-Value stores with very simple APIs.

Caching

- Well designed caching can significantly reduce database load
- Combined with scale up, is sufficient for many applications



Distributed Databases



Distributed Databases – Types

- **Distributed SQL stores**

- Store data across multiple disks that are queried by multiple database engine replicas.
- Multiple engines logically appear to the application as a single database, hence minimizing code changes

- **Distributed so-called NoSQL stores**

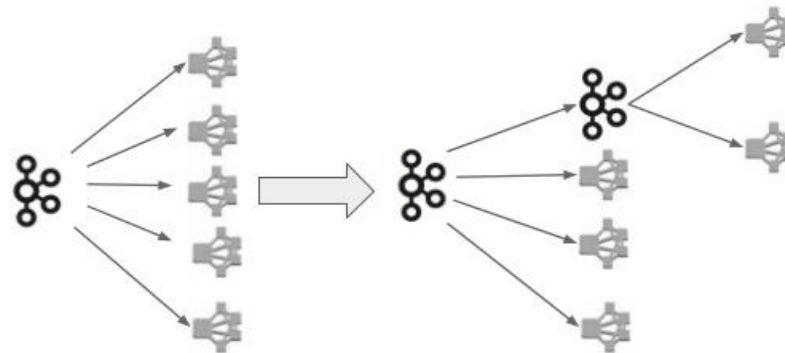
- Variety of data models and query languages
- Distribute data across multiple nodes running the database engine, each with their own locally attached storage.
- Leading products in this category are Cassandra, MongoDB, Neo4j, AWS DynamoDB etc.

Scaling Distributed Databases

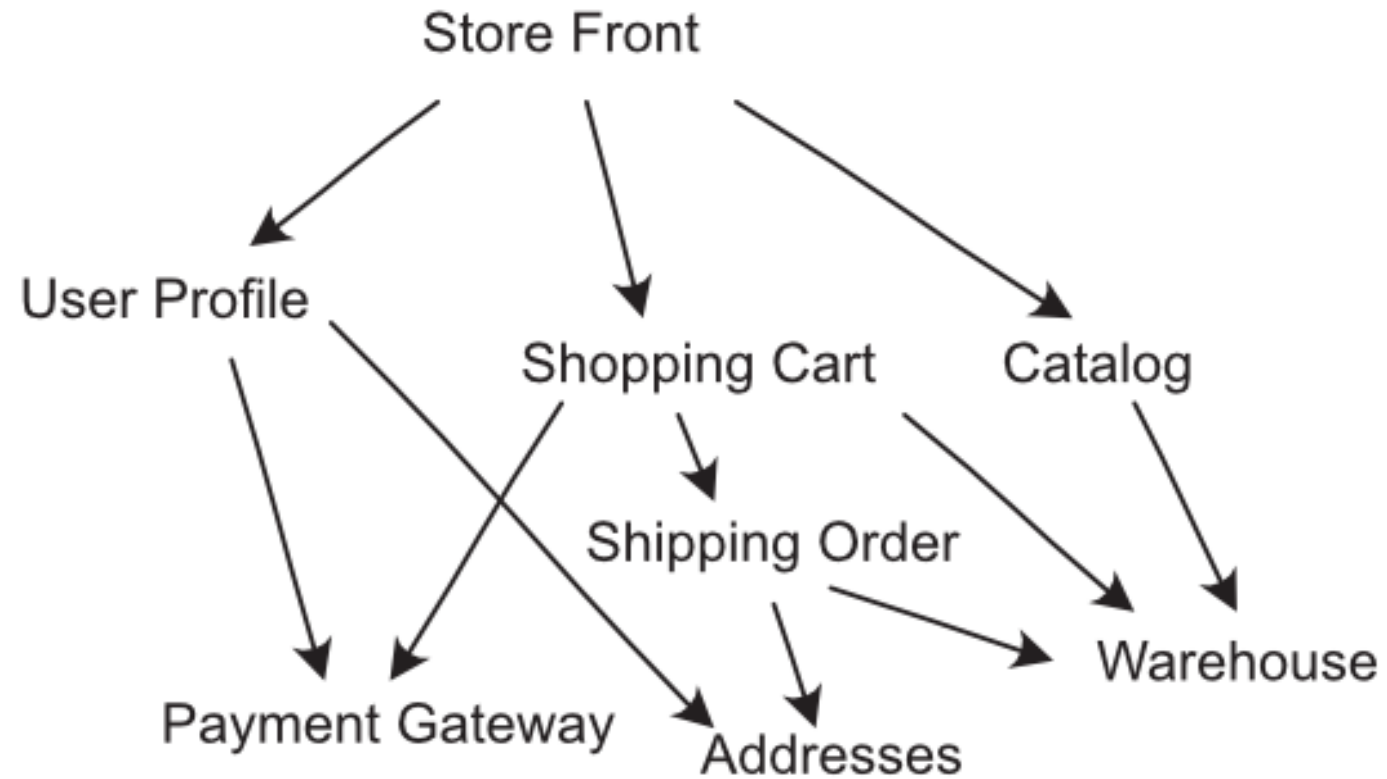
- As data volume grows, the number of database nodes can be **increased**
- Data must be rebalanced to ensure the processing and storage capacity of each node is equally utilized
- Distributed databases also promote **availability** through **replication**
 - If a node fails or cannot be accessed due to network problems, another copy of the data is available
- Can utilize cloud-hosted databases to simplify administration
- Consistency is an issue!!!!

Multiple Processing Tiers

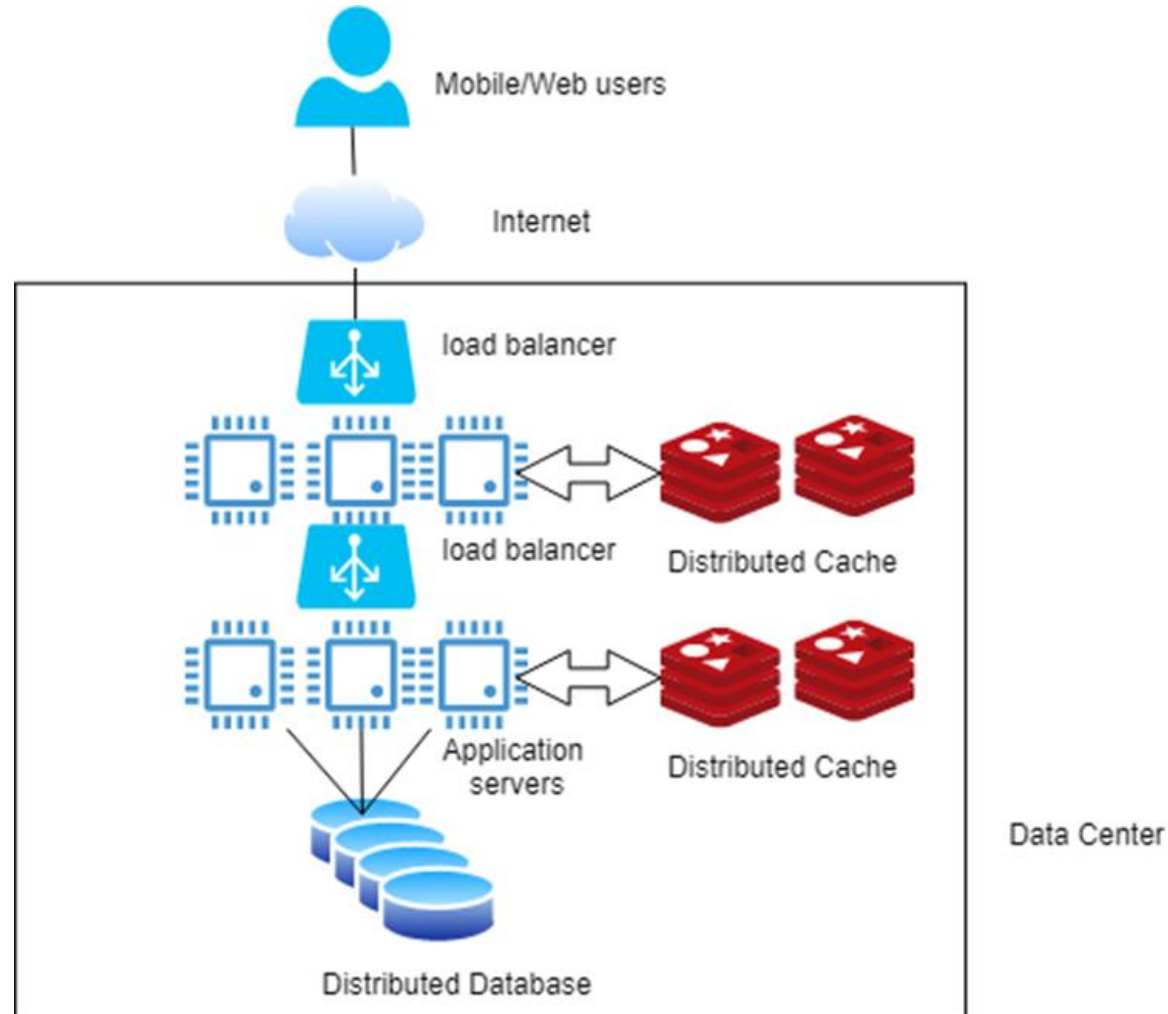
- Any scalable system has many services that interact to process a request
 - Rendering a web page on the Amazon.com requires at least 10,000 different services
- With stateless, cached, load-balanced services, we can extend these core design principles and build a multi-tiered application.
- In fulfilling a request, a service calls one or more downstream services



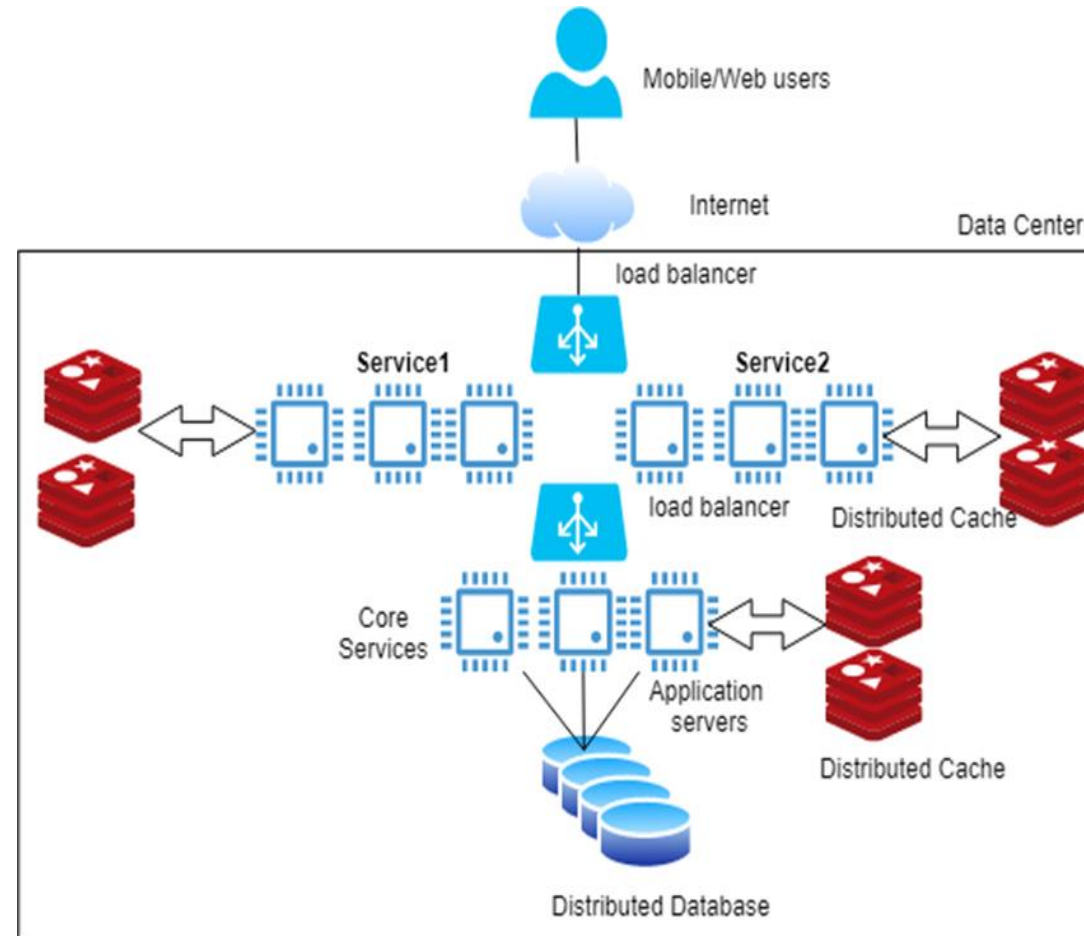
Example of Tiers



Scaling Multiple Processing Tiers



Multiple Processing Tiers

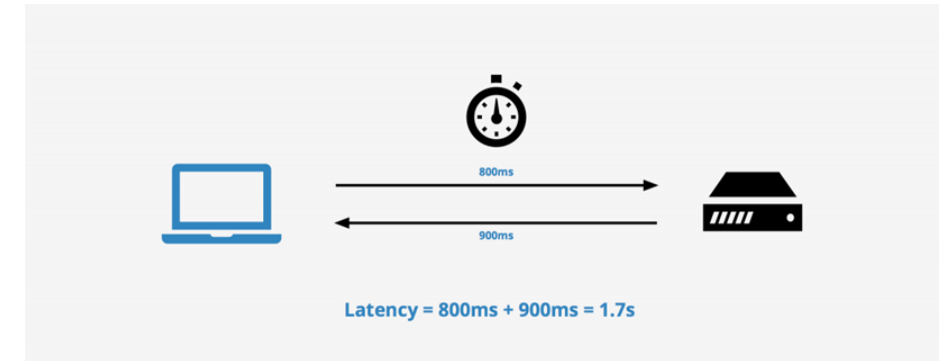


Multiple Services

- Refactoring the monolith into multiple independent services has advantages
 - Easier to build, test, modify
 - Can be scaled individually based on the service demand
- E.g. Two services
 - one for web users
 - one for mobile users
- Can deploy more mobile services and less Web services to meet demand if only mobile usage grows
 - Backend for Frontend (BFF) pattern
 - On the road to microservices

Responsiveness and Latency – Read Requests

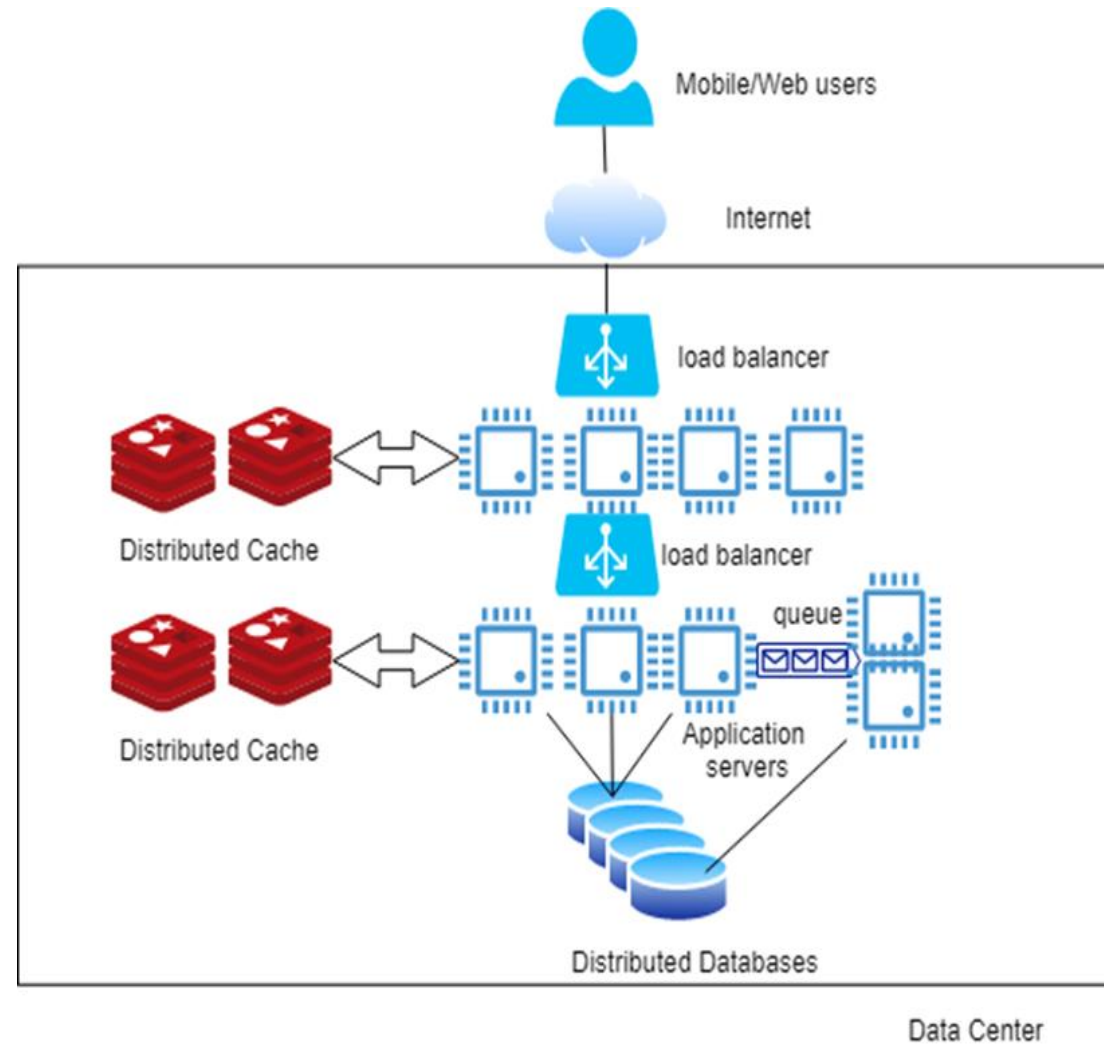
- Most client requests need a response:
 - List all auction items
 - View the real estate for sale in a given location
- Client sends a request and waits until a response is received
- Time interval between sending the request and receiving the result is the **latency** of the request
- Customers care deeply about latency
- Caching and precalculated responses decreases latency
- Many requests will need a (slow) database access



Responsiveness and Latency – Write Requests

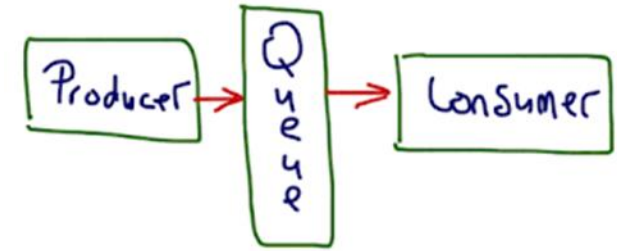
- Most update requests require immediate persistence
 - User updates their default delivery address
 - New delivery address must be persisted so that the user can confirm the address before they hit the 'purchase' button
- Latency includes the time for the (slow) database write
- Some requests don't need to be persisted immediately, e.g.
 - User favorites a product in a catalog
 - User adds a song to playlist
 - Update GPS location
- We can exploit this to improve request responsiveness
 - Client sends request
 - Server responds with a 'promise' to perform the update (no slow database write)
 - Server performs the update 'eventually'

Queueing and Promises

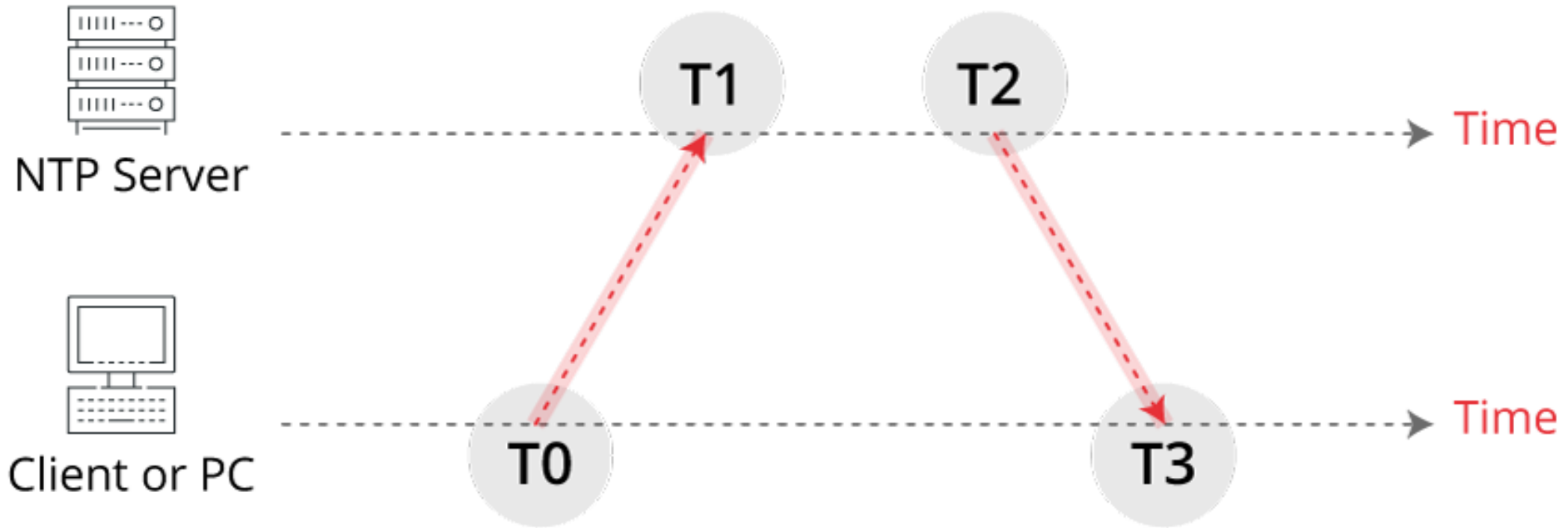


Queueing

- Can be utilized when results of a write are not immediately needed
- Queuing technologies provide asynchronous communications based on the producer consumer pattern
 - Producer writes to queue
 - Consumer reads from queue and updates database
- Data is *eventually* persisted. Eventually means
 - A few milliseconds
 - A second
 - Longer
- Use cases that employ this design should be resilient to longer delays without impacting the user experience

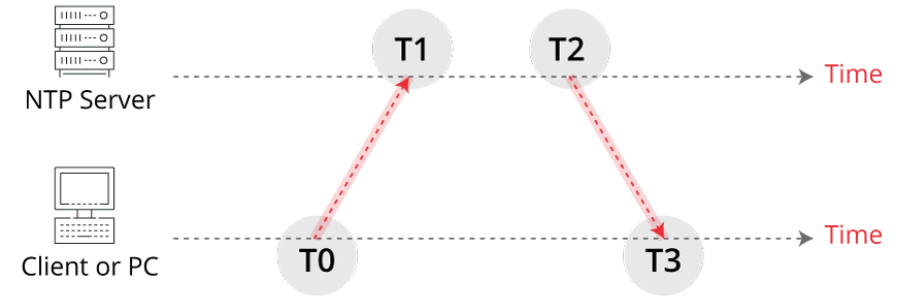


SPECIFICALLY, in Distributed Systems,
Timestamps are problematic!



Timestamps are problematic!

- Last week
 - *Logical clocks!* Big breakthrough! 😊
- But we DO synchronize clocks as best we can...
 - Clock drift means timestamps can't be used to order events...
- But clocks do get generally synchronized!
 - In Linux systems, the Network Time Protocol (NTP) is implemented by a daemon running in user space



Reality...

- Probably not implementing super low level thread coordination much in your day to day jobs?!
 - Cyclic Barrier, Count Down Latch, Semaphores...
 - Dining philosophers, readers/writers, ...
- Even less likely will you be implementing your own 2PC or consensus
 - Paxos, Raft,
 - Blockchain, MongoDB, RabbitMQ, Zookeeper
- But you MIGHT really *enjoy* knowing this is at the core!
 - Practitioners may (will!) disagree with me on this! 😊

Looking back at MapReduce

- A little note under “side effects” ...
 - Overlap of computation and I/O
 - Local writes of intermediate files
 - Remote reads

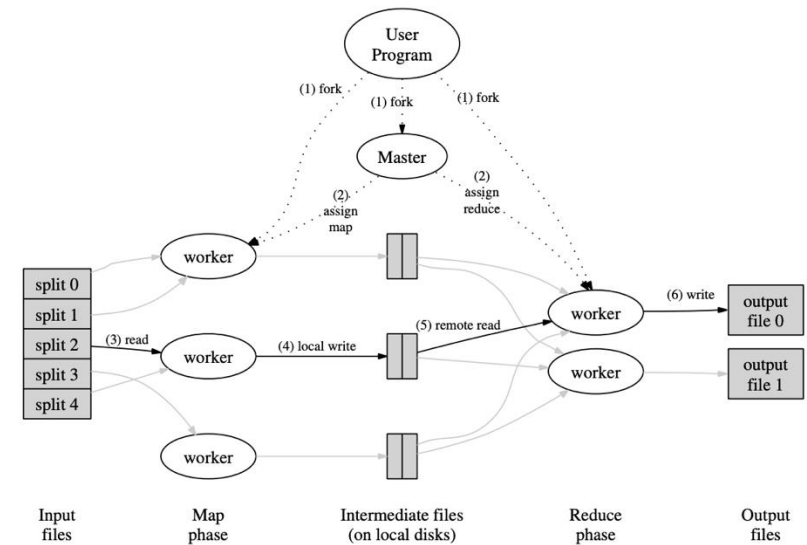


Figure 1: Execution overview

- *“Typically the application writes to a temporary file and atomically renames this file once it has been fully generated. We **do not** provide support for **atomic two-phase commits** of multiple output files produced by a single task. Therefore, tasks that produce multiple output files with cross-file consistency requirements should be deterministic. This restriction has never been an issue in practice.”*

Concurrency

- 3 hard facts of life...
 - Concurrency is necessary for performance
 - why?
 - If you are sharing data with concurrency, you can have consistency problems
 - why?
 - If you are in a distributed system, failures happen!
 - why is that a problem when we can have durable data (on disk)?
 - what does having “logs” on disk have to do with all of this?
 - how do we make things “atomic” across nodes?!?!?
- How do we deal with all of these issues?!

ACID (lessons learned from databases!)

- Atomicity
 - all or nothing!
 - can back out...
- Consistency
 - Every employee must have a salary
 - Consistency constraints means there needs to be some **semantics** for data in transactions!
- Isolation
 - lonely or lovely?!
- Durability
 - *committed* means side effects will persist, even if there is failure
 - main memory for performance but disk/flash for durability

Fair enough!

- Yay databases!
- Now, say we **partition** the data across two databases
 - Let's ignore replication for now
- ACID on *each* database node
 - But how do you commit when multiple databases are hit with operations?!
 - Send a "Please commit!" message to multiple nodes?
 - Get an *ack* back
 - Resubmit if don't get an *ack*...
 - (side joke: why didn't the computer scientist ever get out of the shower?)
 - **Why doesn't this just WORK?!**

What could possibly go wrong?

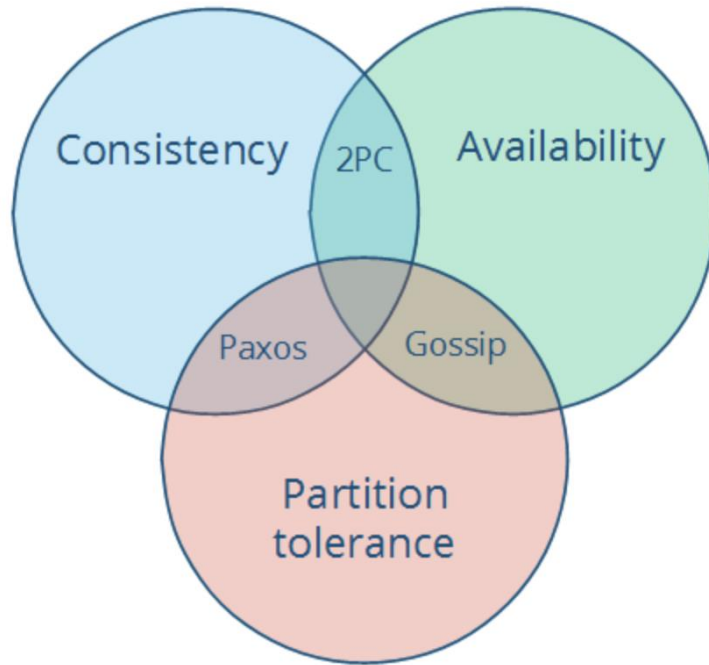
- One of the two databases could say **NO!**
 - Especially rude if the other database already committed!
 - LOST atomicity...
- HOW DO I GET IT ATOMICITY BACK?!
 - 1st phase: “*HEY are you WILLING to commit?*”
 - 2nd phase: Issue a *commit* if they ALL are willing to commit
 - If any node won't commit, then let them all know to abort!
- **ATOMIC:** We need a single point in time when we move from *pre-commit* to *post-commit* for EVERYONE!

2PC

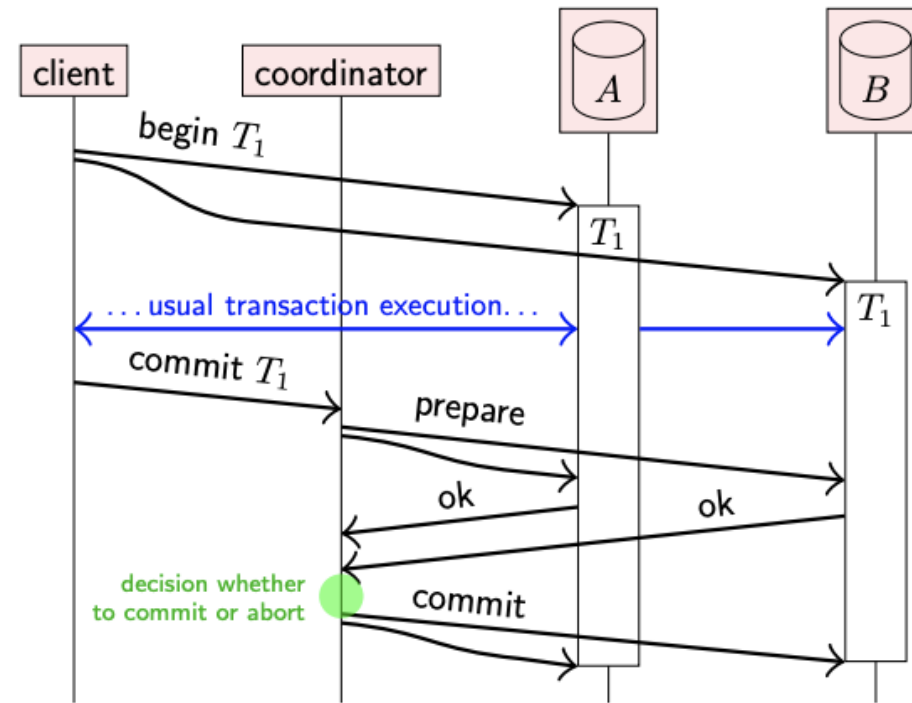
- Coordinator sends out **PREPARE** to all participants
- Participants respond to a PREPARE with **YES** or **NO**
 - If **NO**, immediately abort
- Coordinator collects votes
 - If all were **YES**, (a) coordinator decides to **COMMIT**, (b) sends **COMMIT** to each participant
 - Otherwise (c) decides to **ABORT**, (d) sends **ABORT** to each participant that voted YES
- Participants that voted YES wait for message from coordinator, when received, executes accordingly and protocol stops

Key point in time!

Coordinator needs to make a decision,
Everyone is waiting!!!!



Two-phase commit (2PC)



Properties of an **atomic commit**

1. All processes that reach a decision (commit or abort), reach the same decision (atomicity!)
2. A process CANNOT reverse its decision once it has made it
3. The commit decision cannot be reached unless all processes say “YES”... *Very powerful thing!*
4. If there are no failures and all votes are “YES” then the decision will be to commit
5. After a failure, once all the failures have recovered, *and no new failure happens for sufficiently long*, all processes will eventually reach a decision (*liveness condition*)

WOAH!

- Crazy corner case
 - Coordinator crashes after deciding the vote
 - OUCH!
- What are we going to do?

Precisely defining the problem...

- SO failures happen
- there is an *uncertainty period*
 - I voted **yes**, waiting...
 - I REALLY don't know what is going to happen next!
 - I can't make any more forward progress, can't abort or commit...
- These properties do NOT hold
 - *All processes eventually reach a decision*
 - *All non-faulty processes eventually reach a decision*
 - *The decision to commit will be reached if all processes vote YES*

REALLY important!

- No atomic commitment protocol can *guarantee* forward progress
 - A pattern of repeating failures can prevent nodes from completing the protocol
 - No matter what protocol you use!
- Any atomic commitment protocol requires interaction,
 - which can be delayed arbitrarily long because of failures,
 - this means it might not finish!
- Need to design systems to be aware of this case!

Timeouts in 2PC

- How long should a timeout be?
- Lot of factors... not too long!
- typical interaction time * 2?!
- How will they work here?
- I send a message, start a timer, there will be a timeout if I wait too long!
 - For example, what if a PREPARE never comes? TIMEOUT!
 - Operations are going into write-ahead log...
 - Always safe to abort on a timeout

2PC revisited in gory detail with logs and timeouts!

- Coordinator sends out **PREPARE** to all participants (**log/set timeout**)
- Participants respond to a PREPARE with **YES** or **NO**
 - If **NO**, immediately abort
- Coordinator collects votes
 - If all were **YES**, (a) coordinator decides to **COMMIT (log)**, (b) sends **COMMIT** to each participant (**no timeout!**)
 - Otherwise (c) decides to **ABORT (log)**, (d) sends **ABORT** to each participant that voted YES (**no timeout!**)
- Participants that voted YES wait for message from coordinator (**log/NO timeout!**), when received, executes accordingly and protocol stops

When is the atomic commitment point?

- When the coordinator writes the COMMIT decision to its log!
- This is ONE node
 - If it fails at this point...
 - HOLY COW!
 - We are stuck until the coordinator comes back up!
 - If it does?!?!?
- Can you fix this?!

Distributed Transactions: 2PC + Consensus

- 2PC and Consensus (example, Raft) solve different problems
- All or nothing in 2PC
 - In 2PC different computers to do **different** things
 - e.g. Alice's bank debits Alice, Bob's bank credits Bob
 - and we want them **all** to do their thing, or none of them
- Majority agreement in consensus
 - 3PC (AH!), Paxos, and Raft
 - causes the peers to all do the **same** thing (so they remain replicas)
 - It's OK to wait only for a majority, since the peers are replicas, and therefor we can make the system available in the face of failures.

Consensus

- Originally put in the context of legislatures on an island (Paxos)
 - Leslie Lamport
 - Story is crazy fun...
- Paxos
 - Assures multiple replicas stay in sync
 - Agreement on the order in which operations apply
 - If one does *ab* and the other does *ba* we could be in trouble
 - Turns out this is basically agreement on the value of one variable
 - REALLY? Is that hard?!?!?

Impossibility Result

- Just for the landscape on consensus
 - [https://en.wikipedia.org/wiki/Consensus \(computer science\)](https://en.wikipedia.org/wiki/Consensus_(computer_science))
- FLP Theorem (1985)
 - <https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>
- Need to design systems to deal with this fact!

Check it out!

- Raft talk is posted, but check this out (animation)
 - <http://thesecretlivesofdata.com/raft/>
- Do a walkthrough of the visualization
- It DOES take a while!
 - Leader election
 - Log replication
- A homethink...
 - Describe how the two timeouts are used to control leader elections in Raft!

What's hard about consensus

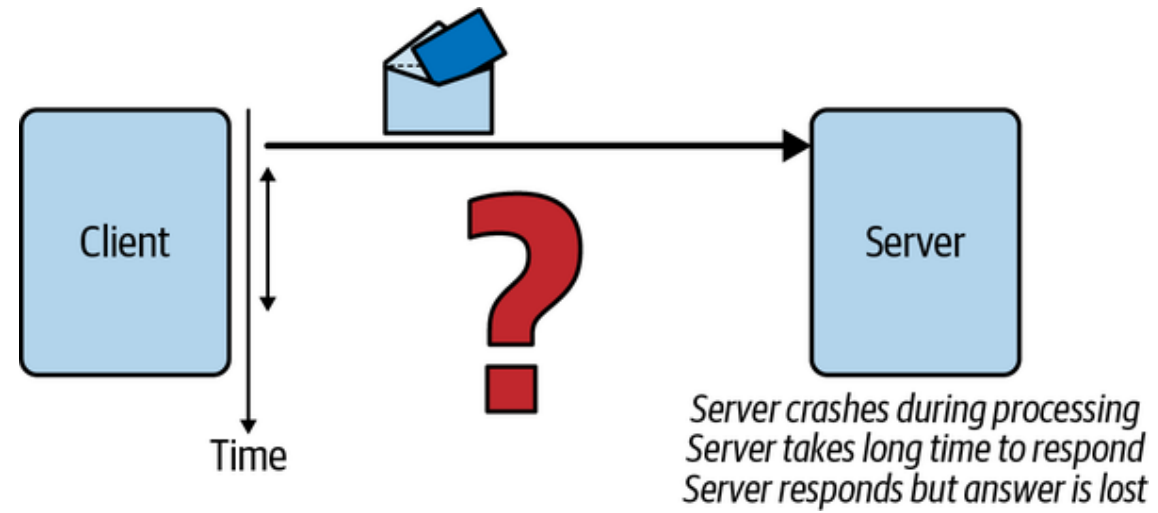
- Two proposers?
 - At the same time propose different values
 - Decide which one wins?
 - Deciding means they need to come to consensus?!?!
- Failure
 - Failure complicates things
 - We need a majority to make a vote work
 - When nodes fail we might not get a majority
- Consensus when you have failures means it might not terminate
- Impossibility result...

Failures in Distributed Systems

Partial Failures

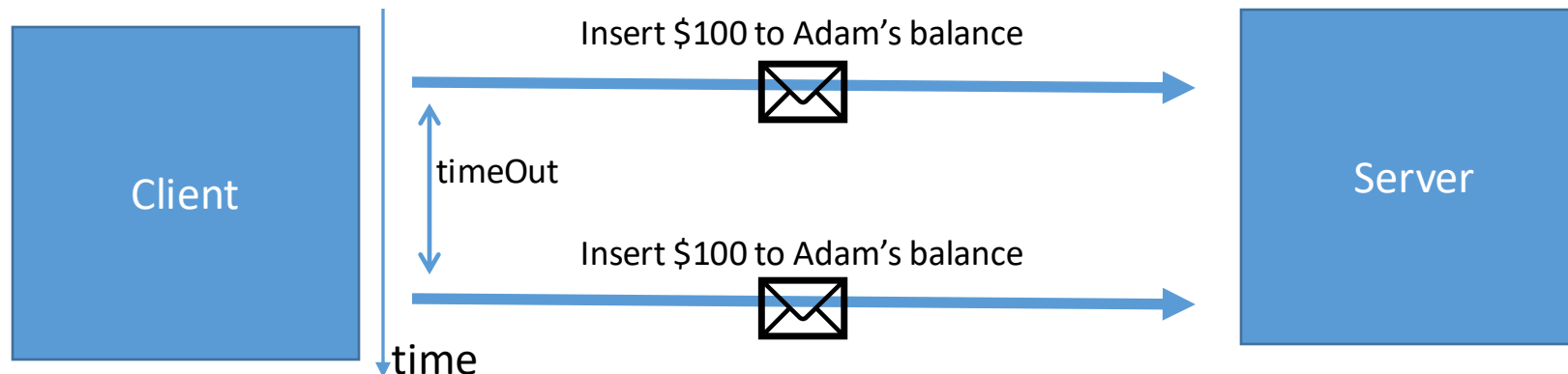
- Distributed applications communicate using an **asynchronous network**
 - Nodes do not have identical internal clocks, hence they are not *synchronized*
 - Nodes can choose to send data to other nodes at *any* time
 - The network is such that one node sends a request and must wait for a response from the other
 - These are two separate communications
 - Time for data to be communicated between nodes is variable - network congestion, transient failures
 - Receiving node may not be available due to a machine crash or other reason
 - Data can be lost due to a variety of reasons

Partial Failures



Partial Failures (2)

- The request succeeds and a rapid response is received.
- The destination IP address lookup may fail. In this case the sender receives an error message. Perhaps the DNS cache is old??
- The IP address is valid but the destination node or target server process has failed. Sender will receive an error message.
- The request is received by the target server, which fails while processing the request and no response is ever sent.
- The request is received by the target server, which is busy. It processes the request but takes a long time to respond
- The request is received by the target server and a response is sent. However, the response is not received by the client due to a network failure.



Distributed Consensus

- *Two Generals* is analogous to nodes reaching agreement on shared state
- Partial failures are analogous to losing messages and acknowledgements!
- It can be demonstrated that consensus is **impossible** to achieve within bounded time
 - On an asynchronous network
 - In the presence of crash faults
 - Messages can be delayed but not lost

Distributed Consensus

- The FLP Impossibility Theorem
 - Michael J. **Fischer**, Nancy A. **Lynch**, and Michael S. **Paterson**. 1985. Impossibility of distributed consensus with one faulty process. J. ACM 32, 2 (April 1985), 374–382.
- FLP is a worst-case scenario
 - Cannot **guarantee** consensus will be reached with **unbounded** message delays on an asynchronous network
- In reality, distributed systems reach consensus every all the time!!!
 - Can establish sensible practical bounds on message delays
 - retry after a timeout period
 - Probability of lack of consensus reduced to **Very Unlikely** – human intervention
 - See **Paxos, Raft**

Byzantine Failures

- Imagine extending the Two Generals problem to N Generals
 - They all need to agree to attack at the same time
 - Traitorous messengers may change the value of the time of the attack
 - Traitorous general may send false information to other generals.
- Malicious failures are known as Byzantine faults
- Algorithms that address malicious behavior exist
 - Blockchain technologies as used by crypto currencies

Summary

- Scalability based on a **few basic approaches**
 - Scale up/out
 - Load balancing
 - Stateless services
 - Caching
 - Distributed databases
 - Queues
- All topics we will revisit throughout the rest of this course!!!